Alessandro Carraro

Edinburgh, UK alecarraro33@gmail.com alecarraro alecarraro.github.io

EDUCATION

University of Edinburgh September 2026 – Present
PhD in Computational Mathematics
Research: AI Model Compression via Structured Linear Algebra
Supervisors: Francesco Tudisco and Desmond Higham

KTH Royal Institute of Technology & Delft University of Technology September 2024 – August 2026
M.Sc. in Computer Simulations for Science and Engineering (Joint Degree)
Cum Laude, GPA: 8.99/10 – Top 3% among Applied Mathematics graduates at TU Delft
Thesis: A Multi-Precision Block-Jacobi SVD for Modern GPUs

Delft University of Technology September 2021 – June 2024
B.Sc. in Aerospace Engineering
Minor in Computational Science and Engineering

EXPERIENCE

Google Summer of Code Contributor, NextLA.jl May 2026 – August 2026
Project: Efficient Exact-Rank Tile-Low-Rank GEMM on GPUs

- Designed a GPU-accelerated Tile-Low-Rank GEMM algorithm with dense accumulation, achieving up to **2.8× speedup** over the existing implementation
- Implemented the algorithm and related variants in `NextLA.jl`, extending the library with GPU-enabled Tile-Low-Rank linear algebra routines

Google Summer of Code Contributor, JuliaReach May 2025 – August 2025
Project: Reachability Analysis for Control of Uncertain Dynamical Systems

- Implemented state-of-the-art reachability algorithms from the recent literature in Julia
- Contributed production-ready numerical algorithms to the open-source `JuliaReach` ecosystem
- Co-authored the ARCH-COMP 2026 Continuous and Hybrid Systems category report

Graduate Teaching Assistant, TU Delft September 2024 — March 2025
Courses in High Performance Computing (Master's and Bachelor's level)

- Assisted students in HPC labs on MPI, OpenMP, and CUDA; graded assignments and reports

RESEARCH

Multi-Precision Block-Jacobi SVD for Modern GPUs January 2026 – June 2026

- Developed and analyzed a mixed-precision block-Jacobi SVD algorithm formulated around GEMM operations to maximize utilization of modern GPU accelerators
- Implemented the method in CUDA C++ and benchmarked performance across NVIDIA H100, and L40 GPUs
- Achieved up to **5× speedup** over a full-FP64 implementation while retaining comparable numerical accuracy on ill-conditioned problems using mixed FP32/FP64 arithmetic

SKILLS

Programming Languages:

- C++ (proficient)
- Julia (proficient)
- Python (intermediate)

Tools: Git, LaTeX

Libraries & Frameworks:

- **GPU Computing:** CUDA C++ (proficient), CUTLASS (basic)
- **Machine Learning:** PyTorch
- **Parallel Computing:** OpenMP

Languages: Italian (native), English (fluent)